



An Axiomatic Concurrency Model





S



-

R  +

R



-



S



—



C



-

R  +

R.B. -



C



—





po

PO







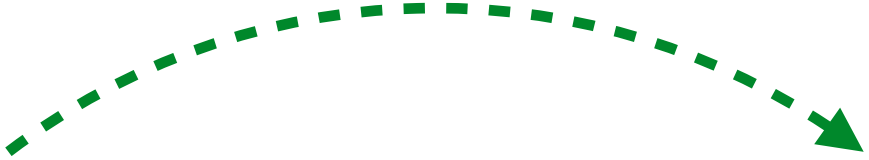
PO

po

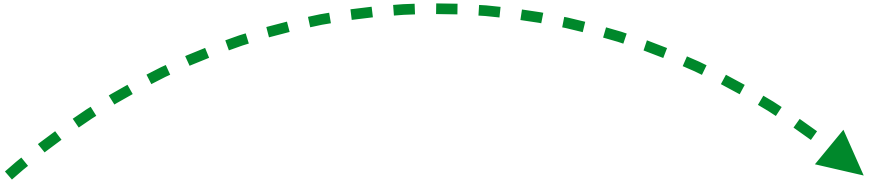


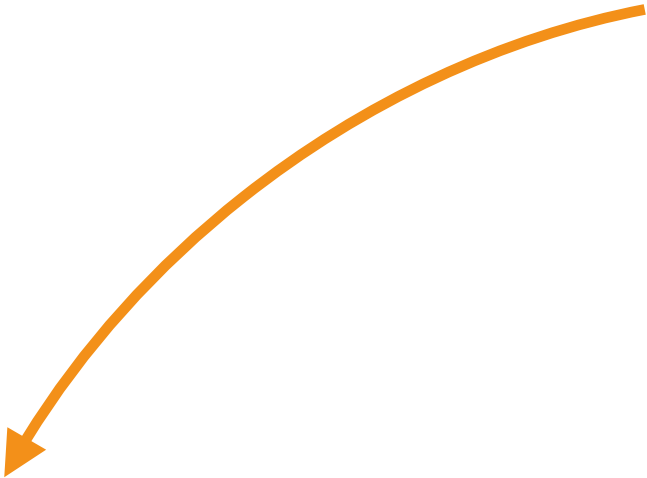
po

po

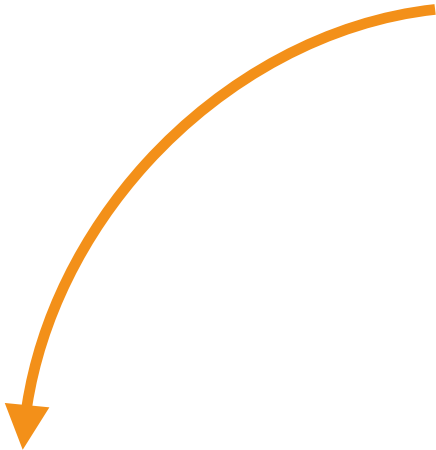




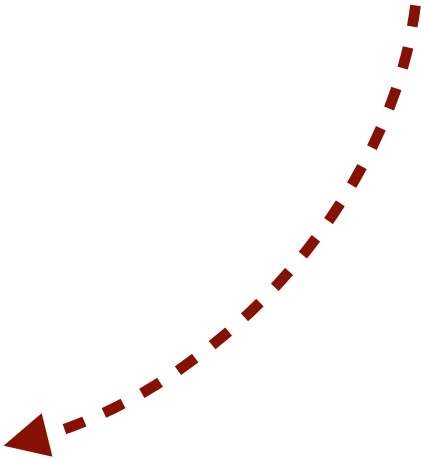


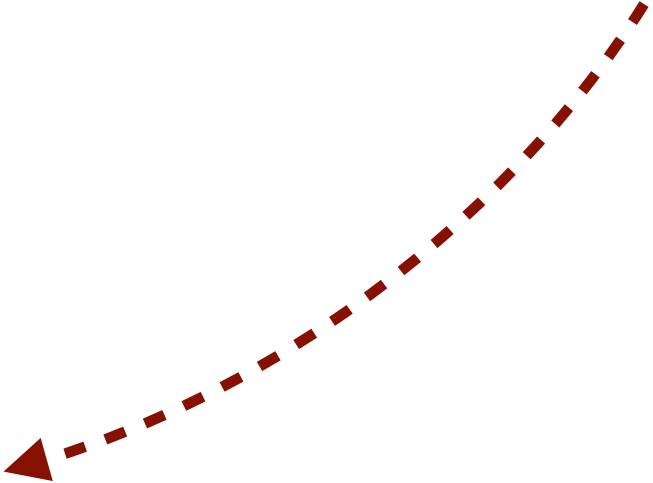


cosrc



collog





hb

h

b

We derive happens before as

```
update(src, +100)
```

```
update(src, -100)
```

$$hb = \{ (C_i, R_j) \mid S_i (po \cup r \cup co)^+ S_j \wedge \\ \text{cowns}(i) \cap \text{cowns}(j) \neq \emptyset \dots \}$$

Is there a causal path between the spawns of two behaviours that require the same cowns?



Candidate executions are tuples of (E, po, co)

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    } } }  
}
```


An Axiomatic Concurrency Model

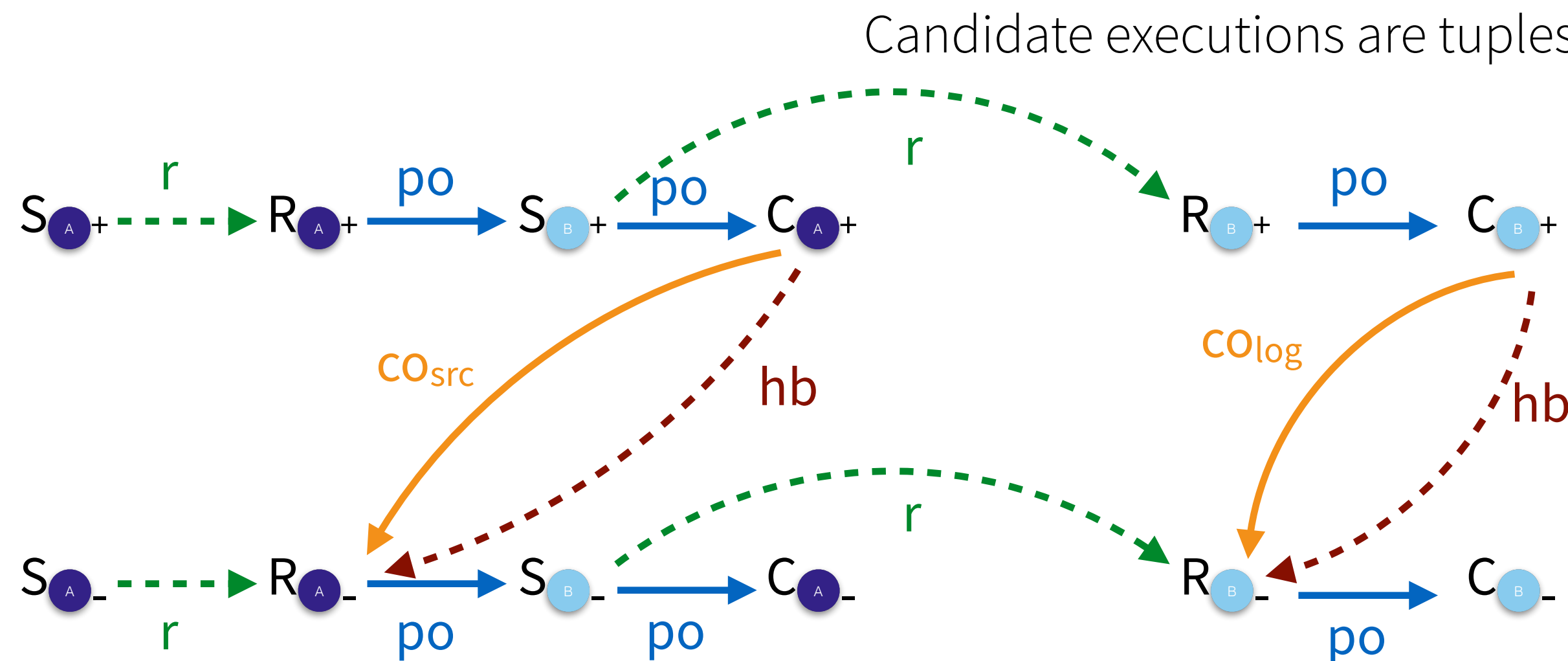
```

def update(account: cown[Account], amount: int) {
  when(account) { A
    if(account.balance + amount > 0) {
      account.balance += amount
      val id = account.id()
      when(log) { B log.record(id, amount) }
    }
  }
}

```

update(src, +100)

update(src, -100)



We derive happens before as

$$hb = \{(C_i, R_j) \mid S_i (po \cup r \cup co)^+ S_j \wedge \text{cowns}(i) \cap \text{cowns}(j) \neq \emptyset \dots\}$$

Is there a causal path between the spawns of two behaviours that require the same cowns?

An Axiomatic Concurrency Model

```
def update(account: cown[Account], amount: int) {  
  when(account) { A  
    if(account.balance + amount > 0) {  
      account.balance += amount  
      val id = account.id()  
      when(log) { B log.record(id, amount) }  
    }  
  }  
}
```

update(src, +100)

update(src, -100)

Valid executions are acyclic in the transitive closure of the union of relations

